CODE DOCUMENTATION

Books Library Code Explanation Guide

دليل شرح كود تطبيق Books Library

This PDF explains the important code across the Flutter frontend and the Python backend. Each entry names the file, explains its responsibility in English and Arabic, and highlights the important logic inside it.

يشرح هذا الملف أهم أجزاء الكود في الواجهة الأمامية المبنية بـ Flutter والخلفية المكتوبة بـ Python. كل جزء يذكر اسم الملف، ويشرح وظيفته بالإنجليزية والعربية، ويبين المنطق البرمجي المهم الموجود بداخله.

2	20+	EN	AR
Main code layers: frontend and backend.	Important files explained in this guide.	English explanation for every important part.	Arabic explanation for every important part.

Prepared on 14 April 2026 as a code-focused companion document for the project.

READING MAP

1. How To Read This Codebase

The project is split into a Flutter client inside `lib/` and a FastAPI backend inside `book_backend/`. Platform folders such as `android/`, `ios/`, `web/`, and `windows/` are configuration targets rather than the core application logic.

Frontend side

The Flutter app handles UI, navigation, local state, API requests, reading screens, search, authentication screens, and audio playback controls.

LIB/

الواجهة الأمامية

تدير واجهة Flutter الشاشات والتنقل والحالة المحلية وطلبات الـ API والقراءة والبحث وشاشات الحساب والتحكم بالصوت.

/LIB

Backend side

The Python backend provides REST endpoints, database access, schema creation, book seeding, content import, and chapter audio generation.

BOOK_BACKEND/

الخلفية

توفر الخلفية المكتوبة بـ Python الواجهات البرمجية، والوصول إلى قاعدة البيانات، وإنشاء الجداول، وبذر الكتب، واستيراد المحتوى، وتوليد الصوت.

/BOOK_BACKEND

Sections in this guide

1. Bootstrapping and global setup
2. Models and data contracts
3. Services and persistence
4. Pages and navigation
5. Reading and audio flow

More sections

6. Reusable widgets and visual layer
7. Backend API and schema
8. Catalog seeding and import scripts
9. End-to-end runtime flow
10. Key engineering ideas

> This guide focuses on the important code. Generated configuration files such as `lib/firebase_options.dart` and the platform folders are not explained in depth because their role is setup rather than core business logic.

BOOTSTRAP

2. Bootstrapping, Theme, and Localization

These files create the app shell, initialize shared state, and make the whole interface bilingual.

`lib/main.dart`

App entry point

English: `main()` initializes Flutter bindings, Firebase, and locally stored favorites before calling `runApp`. `MyApp` wraps `MaterialApp` with locale and favorites scopes so the whole app reacts to language and saved-book changes.

العربية: يقوم `main()` بتهيئة Flutter وFirebase وبيانات المفضلة المحلية قبل تشغيل التطبيق. ثم يلف `MyApp` عنصر `MaterialApp` بنطاقي اللغة والمفضلة لكي تتحدث جميع الشاشات فوراً عند التغيير.

`lib/theme/app_colors.dart`

Central color system

English: This file defines the project color palette in one place. Every page and widget uses these constants, which keeps the UI consistent and makes theme changes easy.

العربية: يجمع هذا الملف ألوان المشروع في مكان واحد. تعتمد الشاشات والودجتات على هذه الثوابت، لذلك تبقى الهوية البصرية موحدة ويسهل تعديل الثيم لاحقاً.

`lib/l10n/app_localizations.dart`

Text translations

English: Stores the English and Arabic text maps and exposes `context.l10n`. This is the main source of translated labels, buttons, messages, and screen text across the app.

العربية: يحتوي على خريطة النصوص بالإنجليزية والعربية ويوفر الوصول إليها عبر `context.l10n`. وهو المصدر الأساسي لكل العناوين والأزرار والرسائل والنصوص داخل التطبيق.

`lib/l10n/app_locale_controller.dart`

Language state controller

English: A lightweight `ValueNotifier` that stores the active language and exposes `toggle()` and `setLocale()`. It is intentionally simple and avoids heavy state-management libraries.

العربية: هو متحكم خفيف من نوع `ValueNotifier` يحفظ اللغة الحالية ويوفر الدالتين `toggle()` و`setLocale()`. تم تصميمه ليكون بسيطاً بدون الاعتماد على مكتبات إدارة حالة كبيرة.

`lib/widgets/language_switch_button.dart`

Runtime language switch

English: This button is reused across pages. It calls `AppLocaleController.of(context).toggle()` and shows the target language label from localization.

العربية: هذا الزر مستخدم في أكثر من شاشة، ويستدعي `AppLocaleController.of(context).toggle()` ليبدل اللغة مباشرة، كما يعرض اسم اللغة القادمة من ملف الترجمة.

MODELS

3. Models and Data Contracts

These files describe the data shape exchanged between the frontend and the backend.

`lib/models/book.dart`

Main book model

English: Represents a book returned by the API. It parses mixed JSON types safely, computes helper values such as `isArabic`, and gives the UI everything it needs to render cards and details.

العربية: يمثل هذا الملف كيان الكتاب القادم من الـ API. كما يعالج اختلاف أنواع البيانات القادمة من JSON بشكل آمن، ويوفر خصائص مساعدة مثل `isArabic` لتسهيل عرض البطاقات والتفاصيل.

`lib/models/chapter.dart`

Chapter structure

English: A small model that holds the chapter identity, owning book, chapter number, and title. It is used by the book details screen, the reader, and the audio screen.

العربية: نموذج بسيط يمثل الفصل ويخزن المعرّف ورقم الكتاب ورقم الفصل والعنوان. يتم استخدامه في شاشة تفاصيل الكتاب والقراءة والصوت.

`lib/models/book_page.dart`

Page content model

English: Holds one readable text unit inside a chapter. The reader and chapter screen use it to show numbered pages and text content.

العربية: يمثل وحدة نصية واحدة داخل الفصل. تعتمد عليه شاشة القراءة وصفحة الفصل لعرض النص مع رقم الصفحة.

`lib/models/catalog_summary.dart`

Dashboard statistics model

English: Stores the summary counts returned by `/books/summary`, such as total books, readable books, English books, and Arabic books.

العربية: يخزن الإحصاءات القادمة من الواجهة `books/summary/` مثل عدد الكتب الكلي والكتب المقروءة والإنجليزية والعربية.

`lib/models/chapter_audio.dart`

Audio response model

English: Represents the backend response for chapter narration. It stores the audio URL, selected voice, provider name, cache status, and chapter metadata.

العربية: يمثل استجابة الخلفية الخاصة بصوت الفصل. ويحفظ رابط الملف الصوتي واسم الصوت المستخدم ومزود الخدمة وحالة الكاش وبيانات الفصل.

SERVICES

4. Frontend Services and Persistence

These files connect the UI to APIs, Firebase, and local storage.

`lib/services/api_service.dart`

HTTP gateway to the backend

English: This is the main networking layer. It chooses the base URL depending on platform, performs GET requests, parses JSON, and exposes typed methods such as `getBooks()`, `getChapters()`, `getPages()`, and `getChapterAudio()`.

العربية: هذا الملف هو البوابة الأساسية للتواصل مع الخلفية. يحدد عنوان الخادم حسب المنصة، وينفذ طلبات GET، ويحول JSON إلى نماذج واضحة مثل `getBooks()` و`getChapters()` و`getPages()` و`getChapterAudio()`.

`lib/services/auth_service.dart`

Firebase authentication wrapper

English: A focused wrapper around `FirebaseAuth`. It exposes sign-in, sign-up, sign-out, the auth state stream, and the current user object.

العربية: يمثل غلافاً بسيطاً حول `FirebaseAuth`، ويقدم عمليات تسجيل الدخول وإنشاء الحساب والخروج، إضافة إلى تيار حالة المستخدم والمستخدم الحالي.

`lib/services/favorites_controller.dart`

Favorite books state

English: Keeps the set of saved book IDs in memory, loads them from storage, updates them when a heart icon is pressed, and exposes them to the widget tree through `InheritedNotifier`.

العربية: يدير مجموعة معرفات الكتب المحفوظة في الذاكرة، ويقوم بتحميلها من التخزين المحلي وتحديثها عند الضغط على القلب، ثم يمررها إلى الشجرة البرمجية عبر `InheritedNotifier`.

`lib/services/favorites_storage_io.dart`

Native favorites persistence

English: Uses `shared_preferences` to save favorite book IDs on native platforms. This is what makes the favorites screen persistent after restart.

العربية: يستخدم `shared_preferences` لحفظ معرفات الكتب المفضلة على المنصات الأصلية، وبذلك تبقى المفضلة محفوظة بعد إغلاق التطبيق وإعادة فتحه.

`lib/services/favorites_storage_web.dart`

Web favorites persistence

English: Stores the same favorite IDs in browser `localStorage` when the app runs on web.

العربية: يحفظ المعرّفات نفسها في `localStorage` الخاصة بالمتصفح عند تشغيل التطبيق على الويب.

`lib/services/favorites_storage_stub.dart`

Fallback storage stub

English: Provides a safe fallback implementation when neither the IO nor web storage backend is available.

العربية: يقدم تنفيذاً احتياطياً آمناً عندما لا تكون آلية التخزين الخاصة بالويب أو المنصات الأصلية متاحة.

SCREENS

5. Navigation and Main Screens

These files define how the user moves through the app and what each main screen does.

`lib/pages/main_shell.dart`

Main navigation shell

English: Holds the floating bottom navigation bar and switches between Favorites, Home, and Search using an `IndexedStack`.

العربية: يحتوي على شريط التنقل السفلي العائم وينقل المستخدم بين المفضلة والرئيسية والبحث باستخدام `IndexedStack`.

`lib/pages/home_page.dart`

Home dashboard

English: Loads books and catalog summary together, builds the statistics cards, and renders featured, readable, English, Arabic, and curated book sections.

العربية: يقوم بتحميل الكتب مع ملخص الفهرس في وقت واحد، ثم يبني بطاقات الإحصاء ويعرض أقسام الكتب المميزة والجاهزة للقراءة والإنجليزية والعربية والمختارة.

`lib/pages/search_page.dart`

Search and filtering

English: Fetches the full catalog once, then filters it in memory by query and by category. This keeps the search interaction responsive on the client side.

العربية: يجلب الفهرس مرة واحدة ثم يقوم بعملية التصفية داخلياً حسب نص البحث والفئة، وهذا يجعل تجربة البحث سريعة على جهة العميل.

`lib/pages/saved_page.dart`

Favorites shelf

English: Reuses the same book data as the home page but filters it to the IDs saved in `FavoritesController`, giving the user a personal shelf.

العربية: يعيد استخدام بيانات الكتب نفسها، لكنه يفلترها حسب المعرّفات المخزنة في `FavoritesController` ليصنع رفاً شخصياً للمستخدم.

`lib/pages/auth_page.dart`

Sign in and sign up UI

English: Contains the email/password form and switches between login and registration modes. It is intentionally compact because the authentication logic lives in `AuthService`.

العربية: يحتوي على نموذج البريد الإلكتروني وكلمة المرور، ويمكنه التبديل بين وضع تسجيل الدخول وإنشاء الحساب. بقي بسيطاً لأن منطق المصادقة الفعلي موجود داخل `AuthService`.

`lib/pages/user_page.dart`

User profile view

English: Reads the current Firebase user and shows basic account information such as email and UID, with a logout action.

العربية: يقرأ المستخدم الحالي من Firebase ويعرض معلومات الحساب الأساسية مثل البريد الإلكتروني والمعرف، مع زر لتسجيل الخروج.

READING FLOW

6. Book Details, Reading, and Audio Screens

These files implement the main user-facing reading experience.

`lib/pages/book_details_page.dart`

Book details decision screen

English: This file is central because it decides what the user can do with a book. It loads chapter data when the book is readable, shows status chips, allows favorite toggling, and routes the user to reading or chapter views.

العربية: يعد هذا الملف محورياً لأنه يحدد ما الذي يمكن للمستخدم فعله مع الكتاب. يقوم بتحميل الفصول للكتب المقروءة، ويعرض شرائح الحالة، ويدير المفضلة، وينقل المستخدم إلى القراءة أو شاشة الفصل.

`lib/pages/book/read_text_page.dart`

Full-text reading page

English: Builds the continuous reading view for a whole book by asking `ApiService` for all chapters and their pages, then rendering them in order.

العربية: يبني واجهة القراءة الكاملة للكتاب من خلال طلب جميع الفصول وصفحاتها من `ApiService` ثم ترتيبها وعرضها بالتسلسل.

`lib/pages/book/chapter_page.dart`

Chapter text and audio player

English: This is one of the most important files in the project. It fetches chapter pages, manages the `AudioPlayer`, tracks playback position and duration, lets the user switch voices, requests audio generation from the backend, and controls play, pause, resume, and stop.

العربية: يعد هذا الملف من أهم ملفات المشروع. فهو يجلب صفحات الفصل، ويدير `AudioPlayer`، ويتابع زمن التشغيل والمدة، ويسمح بتبديل الأصوات، ويطلب توليد الصوت من الخلفية، ويدير أوامر التشغيل والإيقاف المؤقت والمتابعة والإيقاف.

`lib/pages/book/info_page.dart`

Simple information viewer

English: A lightweight page that displays text information blocks using the same visual language as the rest of the app.

العربية: صفحة بسيطة لعرض النصوص والمعلومات الإضافية بنفس اللغة البصرية المستخدمة في بقية التطبيق.

REUSABLE UI

7. Shared Widgets and Visual Layer

These widgets make the UI reusable and keep presentation logic out of the screen files.

`lib/widgets/book_card.dart`

Reusable book card

English: Renders a book in compact or large form, combines cover art, title, author, description, metadata pills, and favorite button, and is reused in Home, Search, and Favorites.

العربية: يعرض الكتاب بصيغة صغيرة أو كبيرة، ويجمع الغلاف والعنوان والمؤلف والوصف والبيانات المختصرة وزر المفضلة، ويعاد استخدامه في الرئيسية والبحث والمفضلة.

`lib/widgets/book_cover_art.dart`

Generated cover artwork

English: Builds visual book covers without depending on downloaded image files. It creates gradients, monograms, badges, and typography based on the book data.

العربية: ينشئ أغلفة بصرية للكتب من دون الاعتماد على صور جاهزة. ويستخدم التدرجات والأحرف المختصرة والشارات والنصوص اعتماداً على بيانات الكتاب.

`lib/widgets/section_title.dart, category_button.dart`

Small presentation helpers

English: These kinds of files are used to keep small repeated UI patterns out of large page widgets, which makes the main pages easier to read and maintain.

العربية: تستخدم هذه الملفات لعزل الأنماط البصرية الصغيرة المتكررة بعيداً عن الشاشات الكبيرة، مما يجعل قراءة صفحات التطبيق وصيانتها أسهل.

BACKEND CORE

8. FastAPI Backend, Database, and Schema Logic

These files are the server-side heart of the project.

`book_backend/main.py`

Main backend application

English: This is the most important backend file. It creates the FastAPI app, enables CORS, serves `/media`, initializes the schema at startup, and exposes all main API routes for books, summaries, chapters, pages, and chapter audio.

العربية: هذا هو أهم ملف في الخلفية. ينشئ تطبيق FastAPI ويفعل CORS ويخدم ملفات `media/` ويهيئ الجداول عند البداية ويقدم جميع الواجهات البرمجية الرئيسية للكتب والملخصات والفصول والصفحات وصوت الفصول.

- Important logic: `get_books()`, `get_books_summary()`, `get_chapters()`, `get_pages()`, `get_chapter_audio()`
- Important helpers: `build_audio_text()`, `detect_tts_language()`, `save_audio_with_fallback()`

`book_backend/database.py`

Database connection

English: Keeps the MySQL connection logic isolated in one file so the rest of the backend can request a connection without duplicating credentials or setup code.

العربية: يعزل منطق الاتصال بقاعدة MySQL في ملف مستقل، حتى تتمكن بقية ملفات الخلفية من طلب الاتصال دون تكرار الإعدادات أو بيانات الاتصال.

`book_backend/catalog_utils.py`

Schema and content utilities

English: Creates the `books`, `chapters`, `pages`, and `chapter_audio` tables, updates older schemas when needed, clears previous book content, replaces chapter data, and resets the catalog.

العربية: ينشئ جداول `books` و`chapters` و`pages` و`chapter_audio`، ويحدث البنية القديمة عند الحاجة، ويحذف محتوى الكتاب السابق، ويستبدل الفصول، ويعيد ضبط الفهرس.

`book_backend/catalog_data.py`

Curated seed data

English: Defines the list of curated English and Arabic books, their descriptions, featured flags, and language codes. This is the source of the seeded library catalog.

العربية: يحدد قائمة الكتب الإنجليزية والعربية المختارة، مع الأوصاف وحالة التمييز واللغة. وهو المصدر الأساسي لفهرس الكتب المزروع في قاعدة البيانات.

IMPORT PIPELINE

9. Catalog Seeding and Book Import Scripts

These files explain how raw source material becomes readable in the app.

`book_backend/catalog_seed.py`

Full catalog seed script

English: Rebuilds the demo catalog from scratch, inserts the curated book list, and imports the two readable titles. It is the best file to read if you want to understand the project dataset lifecycle.

العربية: يعيد بناء فهرس العرض من الصفر، ويدخل قائمة الكتب المختارة، ثم يستورد الكتابين الجاهزين للقراءة. وهو أفضل ملف لفهم دورة حياة بيانات المشروع.

`book_backend/import_book.py`

English JSON import

English: Imports the clean JSON version of *Alice's Adventures in Wonderland*. It repairs broken encoding, normalizes chapter titles, upserts the book record, and writes chapter/page content into the database.

العربية: يستورد النسخة المنظفة من JSON الخاصة بكتاب *Alice's Adventures in Wonderland*. ويصلح الترميز المكسور وينظم عناوين الفصول ويحدث سجل الكتاب ثم يكتب الفصول والصفحات داخل القاعدة.

`book_backend/import_amwaj_akma.py`

Arabic raw-page import

English: Cleans raw Arabic page data, removes noise such as links and mixed-language artifacts, groups pages into sections, exports a generated JSON file, and imports the final structure into MySQL.

العربية: ينظف الصفحات العربية الخام، ويحذف الروابط والضجيج وآثار المزج اللغوي، ثم يجمع الصفحات في أقسام، ويصدر JSON منظماً، ويستورد البنية النهائية إلى MySQL.

> The import scripts are academically important because they show that the project does not only display data; it also transforms and prepares source material before users ever see it.

RUNTIME FLOW

10. End-to-End Code Flow

This section connects the files together so you can explain the system during presentation.

App startup

Files: `lib/main.dart`, `lib/l10n/app_locale_controller.dart`, `lib/services/favorites_controller.dart`

English: The app initializes Firebase and favorites, builds the global scopes, and launches the navigation shell.

العربية: يهيئ التطبيق Firebase والمفضلة، ثم يبني النطاقات العامة ويشغل الهيكل الرئيسي للتنقل.

Browsing books

Files: `lib/pages/home_page.dart`, `lib/pages/search_page.dart`, `lib/services/api_service.dart`, `book_backend/main.py`

English: The frontend requests books and summary data, and the backend returns catalog rows enriched with readability and statistics.

العربية: تطلب الواجهة بيانات الكتب والملخص، وتعيد الخلفية سجلات الفهرس مع حالة القراءة والإحصاءات.

Opening a book

Files: `lib/pages/book_details_page.dart`, `lib/models/chapter.dart`, `book_backend/main.py`

English: Book details request the chapter list, display metadata, and decide whether the user can read the book in-app or only preview it in the catalog.

العربية: تطلب شاشة التفاصيل قائمة الفصول وتعرض البيانات الأساسية وتحدد هل يمكن قراءة الكتاب داخل التطبيق أم أنه مجرد عرض ضمن الفهرس.

Reading text

Files: `lib/pages/book/read_text_page.dart`, `lib/models/book_page.dart`, `lib/services/api_service.dart`

English: The app loads all chapters and pages, then renders them in order as a full reading experience.

العربية: يحمل التطبيق جميع الفصول والصفحات ثم يعرضها بالتسلسل لتكوين تجربة قراءة كاملة.

Listening to audio

Files: `lib/pages/book/chapter_page.dart`, `lib/models/chapter_audio.dart`, `book_backend/main.py`

English: The chapter page asks the backend for narration, the backend generates or reuses an MP3, and the app streams it with audio controls.

العربية: تطلب صفحة الفصل السرد الصوتي من الخلفية، فتقوم الخلفية بتوليد ملف MP3 أو استرجاعه من الكاش، ثم يشغله التطبيق مع أدوات التحكم.

Authentication

Files: `lib/pages/auth_page.dart`, `lib/pages/user_page.dart`, `lib/services/auth_service.dart`

English: The UI delegates account operations to `AuthService`, which forwards them to Firebase Authentication.

العربية: تفوض الواجهة عمليات الحساب إلى `AuthService` الذي ينقلها بدوره إلى Firebase Authentication.

PRESENTATION NOTES

11. Key Engineering Ideas To Mention

If you present the code during your final-year discussion, these are strong points to explain.

1. Clear separation of concerns

UI files focus on presentation, service files focus on communication and persistence, model files focus on data shape, and backend files focus on APIs and data processing.

1. فصل واضح للمسؤوليات

ملفات الواجهة تهتم بالعرض، وملفات الخدمات تهتم بالاتصال والتخزين، وملفات النماذج تهتم بشكل البيانات، أما الخلفية فتركز على الواجهات والمعالجة.

2. Lightweight state management

The app uses `ValueNotifier` and `InheritedNotifier` instead of a heavy framework. This is a good engineering choice for a medium-sized academic project.

2. إدارة حالة خفيفة

يستخدم التطبيق `ValueNotifier` و`InheritedNotifier` بدلاً من إطار ضخم لإدارة الحالة، وهو اختيار مناسب لمشروع أكاديمي متوسط الحجم.

3. Bilingual-first design

The code handles both English and Arabic at the interface level, text direction level, and content level, which makes the project stronger than a single-language demo.

3. تصميم ثنائي اللغة منذ البداية

يتعامل الكود مع الإنجليزية والعربية على مستوى الواجهة واتجاه النص والمحتوى، وهذا يجعل المشروع أقوى من نموذج أحادي اللغة.

4. Practical media pipeline

The audio system is not fake UI. It includes generation, fallback providers, cache reuse, and streaming playback, which is strong evidence of full-stack implementation.

4. منظومة وسائط عملية

نظام الصوت ليس مجرد واجهة شكلية، بل يشمل التوليد ومزوداً احتياطياً وإعادة استخدام الكاش والتشغيل الفعلي، وهذا دليل قوي على تنفيذ متكامل.

Conclusion: if you explain the code in this order, you can move naturally from app startup, to data loading, to reading, to audio, and finally to backend processing and import scripts.